

((گزارش کار فاز اول پروژه کامپایلر))

در فاز اول پروژه کامپایلر، تمرکز اصلی روی طراحی و پیاده‌سازی بخش تحلیل‌گر واژگانی (Lexer) بوده است. این قسمت از کامپایلر وظیفه دارد رشته ورودی زبان برنامه‌نویسی را به توکن‌های معنادار تقسیم کند تا در مراحل بعدی پردازش نحوی (Syntax Analysis) و معنایی (Semantic Analysis) مورد استفاده قرار گیرد. ابزار استفاده شده برای این مرحله، JFlex بوده که یک تولیدکننده تحلیل‌گر واژگانی برای زبان جاوا است و با استفاده از عبارات منظم، توکن‌های مختلف را از کد منبع استخراج می‌کند. (روند اجرا در انتهای توضیحات آمده).

ساختار کلی فایل JFlex به سه بخش تقسیم می‌شود: تنظیمات اولیه، تعریف الگوهای عبارات منظم، و نهایتاً بخش اصلی که الگوها را به کدهای عملیاتی نگاشت می‌کند. در بخش تنظیمات (قرار گرفته بین %% ابتدایی)، نام کلاس خروجی مشخص می‌شود. پشتیبانی از یونیکد فعال شده تا ورودی‌هایی با کاراکترهای خاص نیز پردازش شوند. همچنین متغیرهایی برای دنبال کردن موقعیت هر توکن از جمله شماره خط و کاراکتر تعریف شده‌اند که در خوانایی خروجی کمک می‌کنند. حالات مختلفی برای Lexer نیز تعریف شده‌اند که حالت پایه YYINITIAL و حالت خاص COMMENT برای پردازش کامنت‌ها از جمله مهم‌ترین آن‌ها هستند.

در مرحله بعد، الگوهای مختلف برای شناسایی توکن‌ها تعریف شده‌اند. این الگوها با استفاده از عبارات منظم ایجاد شده‌اند و شامل ساختارهای زبانی زیادی می‌باشند. به عنوان مثال، شناسه‌ها (Identifiers) با الگوی Ident تعریف شده‌اند که از یک حرف آغاز می‌شوند و می‌توانند شامل ترکیبی از حروف، اعداد یا زیرخط (Underscore) باشند. همچنین عددها به انواع مختلفی مانند اعشاری، دودویی و اوکتال تقسیم شده‌اند تا دقت تحلیل‌گر در درک ساختارهای زبانی ورودی افزایش یابد.

عملگرها نیز به طور کامل پوشش داده شده‌اند. این عملگرها شامل عملگرهای ریاضی مانند + و -، عملگرهای منطقی مانند && و ||، عملگرهای مقایسه‌ای مانند == و !=، و همچنین عملگرهای بیتی و انتساب هستند. برای هر گروه از عملگرها، یک الگوی منظم جداگانه در نظر گرفته شده است تا خوانایی و سازمان‌دهی کد بهتر باشد.

پس از تعریف تمامی الگوها، وارد بخش عملیاتی فایل می‌شویم که پس از %% دوم قرار دارد. در این بخش، هر زمان که یک الگو با بخشی از ورودی منطبق شود، یک قطعه کد جاوا اجرا می‌شود. این کد وظیفه دارد نوع توکن، مقدار آن، شماره خط و شماره ستون را چاپ کند. برای تعیین شماره ستون، از اختلاف موقعیت فعلی با موقعیت شروع خط

استفاده می‌شود. تابع `yytext()` مقدار متن توکن را برمی‌گرداند و با استفاده از `yyline` و `yychar`، اطلاعات موقعیت مکانی به دست می‌آید. این اطلاعات برای هر توکن در خروجی چاپ می‌شوند. برای نمونه، اگر توکن `int` شناسایی شود، خروجی به صورت زیر خواهد بود:

یکی از بخش‌های قابل توجه در این پیاده‌سازی، پردازش کامنت‌ها است. کامنت‌ها به دو دسته تقسیم می‌شوند: کامنت‌های تک خطی که با `//` شروع می‌شوند و تا انتهای خط ادامه دارند، و کامنت‌های چندخطی که با `/*` آغاز و با `*/` پایان می‌یابند. برای مدیریت کامنت‌های چندخطی، تحلیل‌گر وارد حالت `COMMENT` می‌شود. در این حالت، محتوای کامنت در یک `StringBuilder` ذخیره می‌شود و پس از مشاهده `/*`، خروج از حالت کامنت انجام شده و محتوای کامنت همراه با اطلاعات خط و ستون چاپ می‌شود. این ساختار از بروز خطاهای تحلیل درون کامنت‌ها جلوگیری می‌کند و باعث می‌شود کامنت‌ها همان‌طور که انتظار می‌رود نادیده گرفته شوند.

در این پروژه، علاوه بر شناسایی دقیق توکن‌ها، سناریوهای مختلفی برای مدیریت خطا نیز در نظر گرفته شده است. به عنوان مثال، اگر یک رشته (`String`) شروع شود ولی هیچ‌گاه بسته نشود، پیامی تحت عنوان `"Unterminated string"` چاپ می‌شود. همچنین در صورت مواجهه با کاراکترهای ناشناخته یا توکن‌های نامعتبر، پیام خطا همراه با موقعیت دقیق آن در خروجی ثبت خواهد شد.

روند اجرا:

برای مثال در فایل ورودی که در فایل پروژه موجود است، این ورودی:

```
define PI 3.14159#
```

```
define MAX_VALUE 100#
```

```
<template <typename T
```

```
    } class MathUtils
```

```
public:
```

خروجی زیر را داده است:

```
Run compilerFaz1 x
"C:\Program Files\Java\jdk-19\bin\java.exe" "-javaagent:C:\Users\Pedram\AppData\Lo
Preprocessor Directive (#define) - Line 1, Column 1
Identifier (PI) - Line 1, Column 9
Float Literal (3.14159) - Line 1, Column 12
Preprocessor Directive (#define) - Line 2, Column 1
Identifier (MAX_VALUE) - Line 2, Column 9
Integer Literal (100) - Line 2, Column 19
Keyword (template) - Line 4, Column 1
Comparison Operator (<) - Line 4, Column 10
Identifier (typename) - Line 4, Column 11
Identifier (T) - Line 4, Column 20
Comparison Operator (>) - Line 4, Column 21
Keyword (class) - Line 5, Column 1
Identifier (MathUtils) - Line 5, Column 7
Separator ({) - Line 5, Column 17
Keyword (public) - Line 6, Column 1
```

که به تفکیک هر بخش را مشخص کرده و خط و ستون را هم نشان میدهد. در انتهای فایل ورودی هم این ورودی:

```
this is comment singel line //
```

```
this is comment */
```

```
/*multiple line
```

```
int x = 10; // this comment is end of line
```

```
;return 0
```

```
{
```

خروجی زیر را داده است:

```
33 comparisonOperator = "<=" | ">=" | "!=" | ">"
34 logicalOperator = "&&" | "||" | "!"
35 bitOperator = "<<" | ">>" | "~" | "^"
36 assignmentOperator = "=" | "+=" | "-=" | "*=" | "/="

Run compilerFaz1 x

Single Line Comment (// this is comment single line) - Line 40, Column 0
Multi Line Comment (/* this is comment
    multiple line*/) - Line 49, Column 5
Keyword (int) - Line 51, Column 5
Identifier (x) - Line 51, Column 9
Assignment Operator (=) - Line 51, Column 11
Integer Literal (10) - Line 51, Column 13
Separator (;) - Line 51, Column 15
Single Line Comment (// this comment is end of line) - Line 51, Column 17
Keyword (return) - Line 53, Column 5
Integer Literal (0) - Line 53, Column 12
Separator (;) - Line 53, Column 13
Separator (}) - Line 54, Column 1

Process finished with exit code 0
```

که همانطور که مشاهده میکنید کامنت چندخطی را نیز به درستی تشخیص داده است. فایل کامل ورودی در پروژه موجود است و با اجرای کد میتوانید خروجی کامل را نیز مشاهده کنید.